



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

In my implementation, I understood the State Space as all the possible positions that the agent can reach in the grid while considering the walls and grid boundaries. The Search Tree is the way the search algorithm explores those possible states starting from the agent's current position. For example, when I ran BFS, it expanded the possible grid positions level by level from the starting position. So, the State Space represents the possible states in the environment, while the Search Tree represents how the algorithm explores those states to find a path to the food.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

In my implementation, I used the `reached` set to keep track of the grid positions that had already been explored. This prevents the search agent from visiting the same state repeatedly and getting stuck in cycles. This was especially important for my DFS implementation because DFS can move deeply through one path and then come back to previously visited positions. Without the `reached` set, the DFS agent could repeatedly move between the same states, such as $A \rightarrow B \rightarrow A \rightarrow B$, and could potentially continue searching indefinitely instead of finding the food.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

When I tested both algorithms in my simulation, I observed that BFS generally followed a more direct path to the food, while DFS explored deeper paths and could take a winding route. In my particular run, BFS completed the simulation in 29 steps with a score of 200, whereas DFS reached the 60-step limit with a score of 100. BFS is optimal in this grid because every movement has the same cost, so BFS explores the states level by level and reaches the goal using the minimum number of moves. DFS explores one branch deeply before trying other branches, so it can find a path without guaranteeing that the path is the shortest one.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

From my implementation, I understood that BFS requires much more memory because it keeps many unexplored nodes in its FIFO frontier at the same time. Its space complexity is $O(b^d)$, where b is the branching factor and d is the depth of the shallowest goal. Therefore, on a 1000x1000 grid, the BFS frontier and reached set could become very large and cause a memory problem. DFS uses a stack and mainly stores the current deep path and remaining alternatives, giving a space complexity of $O(bm)$, where m is the maximum search depth. Therefore, DFS is generally more memory-efficient than BFS, although it may take a longer and non-optimal route.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING